GIK Institute of Engineering Sciences and Technology, Topi, Swabi, Khyber Pakhtunkhwa, Pakistan

Semester Project Proposal

Object Oriented Programming

CS – 112

Maze Runner – A Text-Based Maze Game Using OOP

Submitted by:

Muhammad Daniyal - 2024373

Shayan Ahmad Hussain - 2024584

Zohaib Ali - 2024687

Submitted to: Dr Sarah Iqbal

Submission Date: 14th March,2025

Introduction:

Maze Runner is a console-based C++ game where the player navigates through a 2D maze using keyboard inputs. The goal is to move from the starting position to the finish point, marked as 'F', while avoiding walls (#) and optional traps (T). The maze is displayed using ASCII characters in the console and updates visually after every move. The game also tracks the number of moves taken and the total time the player spends in the maze. The game ends when the player either reaches the finish or chooses to exit manually.

Objectives:

- To design and develop a console-based maze game using C++
- To apply core object-oriented programming (OOP) principles such as:
    - Encapsulation
    - Inheritance
    - Polymorphism
    - Information Hiding
- To allow the player to interactively navigate a 2D maze using keyboard inputs.
- To track player performance through move count and elapsed time
- To practice problem-solving, logic implementation, and user-focused design.
- To provide a simple and fun user experience using text-based UI.

Key Features:

- Player moves through a maze using keyboard inputs.
- The maze updates visually in the console using ASCII characters.
- A timer tracks how long the player takes to complete the maze
- The game records the number of moves made by the player.
- No GUI – only text-based console interaction.

Name & Score Saving (File I/O):

- At the start, the player enters their name
- At the end, their name, time taken, and move count are stored in a text file
- This allows for high score tracking or viewing top players

Features of the Maze

- **Walls (#)** – Block movement and form the maze structure - *implementing*
- **Traps (T)** – Penalize the player (e.g., reset position or reduce life) - *optional*
- **Treasures ($)** – Bonus objects that increase score- *implementing*
- **Enemies (E)** – Move randomly; reset player on contact - *implementing*
- **Exit Cell (F)** – Reaching this ends the game - *implementing*
- **Custom Maze** – Players can play different maze layouts – *optional*

Literature Review:

Before choosing this game, we explored various project ideas such as management systems and simple games like Tic Tac Toe. However, many of these options either lacked creativity or didn't allow us to fully demonstrate core OOP ideas like inheritance, encapsulation, and polymorphism in a meaningful context. Our approach focuses on keeping the game text-based and simple, while making it interactive and engaging. We use basic grid-based logic to simulate movement and ASCII characters to represent the maze visually in the console. The decision to add tracking of moves, time, and player name with score saving was inspired by common arcade-style games such as Pacman, Snake, and old DOS-based games that were popular among school computer labs in Pakistan. These types of games offered simple visuals but rewarding gameplay through high-score tracking and challenge-based progression — a concept we aim to recreate with our Maze Runner project.

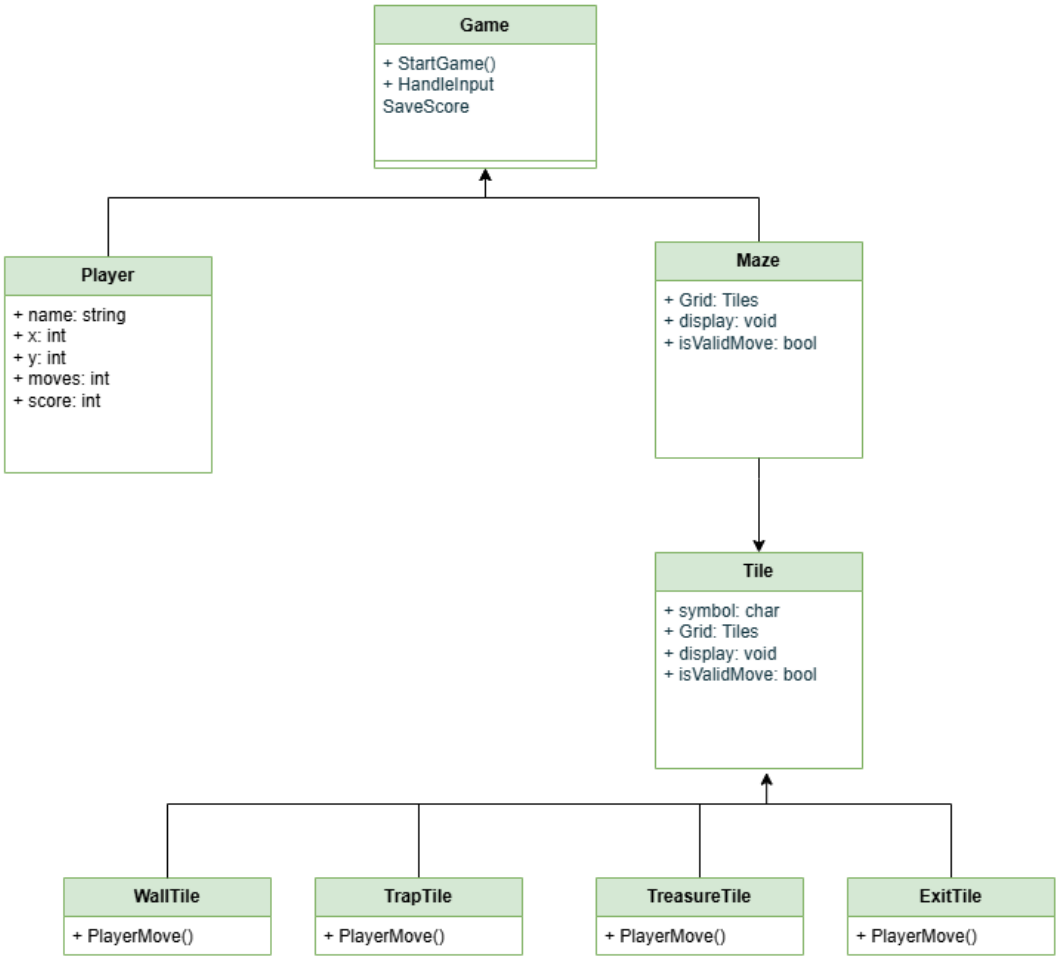
Basic Entities Identified

These are the core, real-world concepts we based our design around:

- **Player** – The user navigating through the maze

- **Maze/Grid** – The environment or map the player moves in
- **Wall** – A solid block that cannot be passed
- **Path** – Open space the player can walk through
- **Trap** – Penalizes the player when stepped on (optional)
- **Treasure** – Gives the player extra score (optional)
- **Enemy** – Moves randomly, resets the player (optional)
- **Exit/Finish** – Marks the end point of the maze
- **Game** – Controls the flow of everything

Class Diagram:



Key Components:

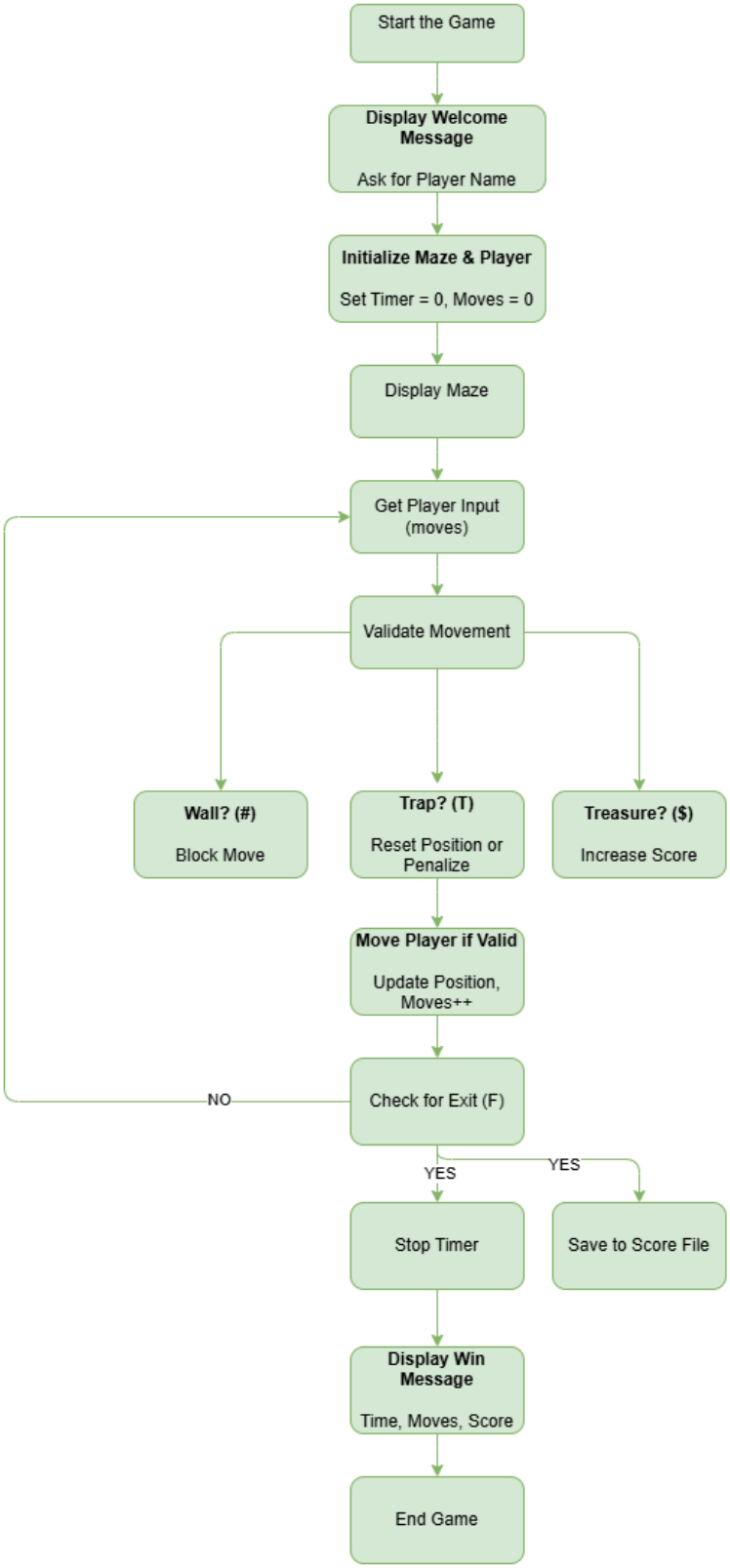
- **Game Class**
  - Manages the overall game flow: starting the game, handling input, checking win conditions, and saving scores.
  - Composes the Player and Maze classes.
- **Player Class**
  - Stores player-related data: name, position (x, y), move count, and score.
  - Handles movement and score updates.
- **Maze Class**
  - Stores and manages a 2D grid of Tile objects.
  - Responsible for displaying the maze and validating player movement.
- **Tile (Abstract Class)**
  - Base class for all maze elements.
  - Defines a virtual method PlayerMove() for tile-specific behavior.

Inherited Tile Types:

- **WallTile** – Blocks player movement
- **TrapTile** – Resets or penalizes the player
- **TreasureTile** – Increases player score
- **ExitTile** – Ends the game upon entry

Each tile type overrides the PlayerMove() method to define its interaction with the player, allowing polymorphic behavior.

Flowchart:

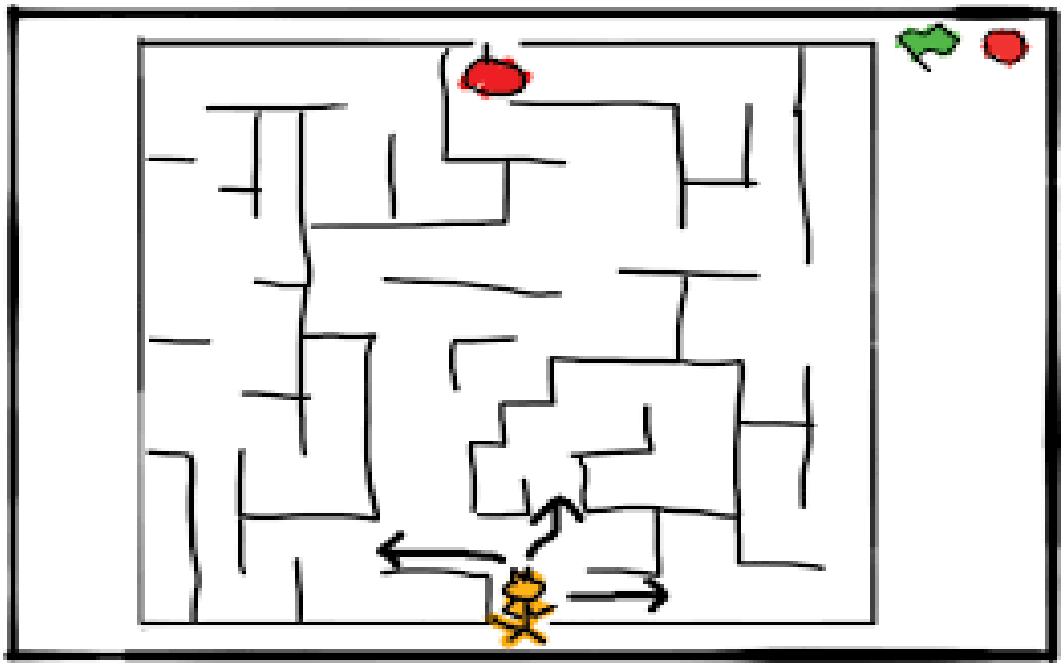

Start the Game
Display Welcome Message
Ask for Player Name
Initialize Maze & Player
Set Timer = 0, Moves = 0
Display Maze
Get Player Input (moves)
Validate Movement
Wall? (#)
Block Move
Trap? (T)
Reset Position or Penalize
Treasure? ($)
Increase Score
Move Player if Valid
Update Position, Moves++
Check for Exit (F)
NO
YES
YES
Stop Timer
Save to Score File
Display Win Message
Time, Moves, Score
End Game

OOP Concepts Used:

- **Classes & Objects:**
  - To model the game entities such as Player, Maze, Game, and different types of maze elements (Wall, Trap, Treasure, Enemy, Exit).
  - Each class will encapsulate its own data and behaviors.
- **Encapsulation:**
  - Player details (name, position, moves, score, etc.) will be kept private within the Player class.
  - Maze layout and tile logic will be hidden inside the Maze class.
  - Public methods will be used to safely interact with internal data.
- **Inheritance:**
  - A base class Tile will be created to represent a generic cell in the maze.
  - Specific elements like Wall, Trap, Treasure, Exit, and Enemy will inherit from Tile and override its behavior.
- **Polymorphism:**
  - We will use polymorphism to allow different maze elements (like walls, traps, enemies, etc.) to behave differently when the player interacts with them, while still being treated as the same general type.
- **Abstraction:**
  - Abstraction will help us hide the complex inner workings of the game (such as movement validation, event handling, or file operations) behind simple, easy-to-use functions. This will make the code easier to work with and understand as the project grows.
- **File Handling (File I/O):**
  - Player name, number of moves, and time taken will be saved to a text file at the end of each game.
  - This allows for storing and displaying high scores.
  - (Optional) File input may be used to load custom maze layouts or player progress.

Creativity / Novelty:

Although Maze Runner is a text-based game, we've added several creative features to make it more fun and interesting. Instead of just moving through a maze, the player will interact with things like traps**,** treasures, and enemies, each doing something different. This makes the game feel more dynamic and challenging. We're also using file handling to save the player's name, score, and time, so we can keep a high score record. This adds a competitive element to the game. Features like custom maze layouts and random enemy movement can be added later to make the game more re-playable. Overall, the project not only shows OOP concepts clearly, but also aims to give a fun and engaging experience.



Conclusion:

Maze Runner is a beginner-friendly yet creatively designed project that allows us to apply core object-oriented programming concepts in a fun and interactive way. Through features like real-time maze navigation, move and time tracking, file-based score saving, and optional game elements such as traps, enemies, and treasures, we aim to demonstrate both technical understanding and creative thinking.